

---

## Assignment 3: Soft Actor Critic

---

Rejdi Danaj s3988651<sup>1</sup> Manuel Pérez Belizón s4416880<sup>2</sup> Harshad Sansanwal s3966003<sup>3</sup>

### Abstract

We discuss the Soft-Actor-Critic (SAC) algorithm, highlighting its theoretical foundations, implementation details and experimental performance within the Cartpole-V1 environment. SAC deploys an entropy term to help facilitate exploration and improving learning stability across random seeds. Experimental evaluations conducted with varying entropy coefficient ( $\alpha$ ), demonstrate that higher values significantly improved the performance by encouraging wider policy exploration. Addition of the double-Q trick further improved the results. Though despite gains, the instability during late stages in training caused by high entropy suggests the potential benefit of automated entropy tuning. Limitations of this study include the simplicity of the CartPole environment and computational resource constraints, indicating directions for future work involving more complex environments and automated entropy-tuning mechanisms.

## 1. Introduction

This report introduces reader to the Soft Actor Critic (SAC) algorithm. SAC implements similar strategy as actor-critic methods for loss functions with an additional strategy to facilitate exploration to avoid pitfalls of local maxima for stochastic policy optimization. This is achieved by adding the entropy information of the policy distribution to the loss function to enforce the entropy to stay large. Through this report, we will explain our motivations behind the choice of employing SAC for the Cart-Pole environment. Also walk you through the SAC algorithm, the implementation of it and our test results achieved.

## 2. Theory

### 2.1. Motivation of choice

Between the two algorithms Soft Actor Critic and Proximal Policy Optimization the choice for implementing SAC is motivated by two main factors. First, with ensured large policy distribution entropy during learning the SAC facilitates exploration in a continuous state and allows stability

by avoiding local maxima. Additionally, by including entropy in the loss function we are also able to ensure stable results across different random seeds and reduced sensitivity to hyperparameters. Second, SAC being an off-policy method and utilizing replay buffer (storing past sampled transitions) provides considerably higher sample efficiency. (Plaata, 2022)

### 2.2. Loss function explained

The SAC loss functions builds upon ideas from the standard Actor-Critic methods. But it extends upon these approaches by incorporating entropy and using off-policy data (Haarnoja et al., 2018) & (Plaata, 2022). In SAC, similar to AC we use two interconnected loss functions for the actor (policy) and the critic (value estimator). The critic in SAC aims to minimize the soft Bellman equation, while the actor aims to maximize the expected reward with the policy entropy.

**The critic loss** is a mean squared error loss between the current Q-value for state  $s$  and action  $a$  and the target Q-value  $y$  calculated using the Q network:

$$\mathcal{L}_Q(\phi) = \mathbb{E}_{(s,a,r,s',d) \sim \mathcal{D}}[(Q_\phi(s,a) - y)^2] \quad (1)$$

The target Q-value is calculated as the addition of the current reward plus the predicted Q-value for the next pair of state & action ( $s', a'$ ) and the entropy of the policy. We employ two Q-functions to counter for any overestimation in predicted target Q-value (clipped double-Q method), by using the minimum of their respective predictions to compute the target:

$$y = r + \gamma(1 - d)(\min_j Q_{\phi_j}(s', a') - \alpha \log \pi_\theta(a'|s')), \quad (2)$$

$$a' \sim \pi_\theta(\cdot|s')$$

**The actor loss** directly incorporates the entropy into the policy objective. The goal is to update the gradient to nudge the objective towards action with highest reward adjusted by policy entropy. The entropy term helps promoting exploration through uncertainty and enabling robust policies.

The loss function here is already written in negative form to consider for minimization of the loss function, implying maximization:

$$\mathcal{L}_\pi(\theta) = \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_\theta(\cdot|s)} [\alpha \log_\theta(a|s) - \min_j Q_{\phi_j}(s, a)] \quad (3)$$

Where,

$\alpha \leftarrow$  entropy temperature

The loss function utilized in SAC can be seen as a natural extension of the Actor-Critic general policy gradient, with the addition of policy entropy. As mentioned in (Plaat, 2022) this general gradient is formulated as:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau_0 \sim p_\theta(\tau_0)} \left[ \sum_{t=0}^n \psi_t \nabla_\theta \log \pi_\theta(a_t | s_t) \right] \quad (4)$$

Where,

$\tau \leftarrow$  trajectories;

$\psi_t \leftarrow$  choice of value function estimator

Now if we look at the gradient of the loss function mentioned for SAC in Eq(3) and set  $\alpha = 0$  to remove the entropy term. We get a similar policy gradient as the general one Eq(4), where value function estimator is  $\psi \rightarrow Q_\phi(s, a)$ :

$$\nabla_\theta L_{SAC} = -\mathbb{E}_{a \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a|s) \cdot Q_\phi(s, a)] \quad (5)$$

### 2.3. SAC Algorithm

Now to further explain how the SAC algorithm works, we will provide detailed steps for Algorithm 1, which is motivated by previous work (Haarnoja et al., 2019):

1. To begin, the agent observes the current state  $s$ , samples an action from the current policy  $\pi_\theta$ , executes the action and obtains a reward  $r$ , next state  $s'$ , and termination signal  $d$ . These are all then stored in the replay buffer.
2. Updates on the initialized policy and value networks occur only every  $K$  steps and only after the replay buffer has collected enough samples  $\geq B$ .
3. For training our networks, we select a mini-batch of traces from the replay buffer at random.
4. Before we calculate the expected soft state value, we calculate the element-wise critic prediction for the two critic networks  $Q_{\phi_1}, Q_{\phi_2}$ . Choose the minimum estimate  $q^{\min}(s', a')$  to counter overestimation.

5. Now we use  $q^{\min}(s', a')$  to calculate the expected soft state value  $V_{\text{soft}}(s')$  by weighting the Q-value with action probabilities and subtracting the entropy penalty  $\alpha \log \pi_\theta$ .
6. The discounted soft value  $\gamma V_{\text{soft}}(s')$  is then used to calculate the critic training targets by adding the current reward. Also, ensuring no bootstrapping occurs at terminal states ( $d = 1$ ).
7. Now we can adjust both the critics by minimizing the squared error between the current predictions  $Q_{\phi_j}(s, a)$  and computed target  $y$ .
8. Finally improve the policy parameters by maximizing the expected returns, while balancing the predicted Q-value  $q^{\min}(s', a')$  and the policy entropy  $\alpha \log \pi_\theta$  ensuring the policy search stays as wide as possible.

---

**Algorithm 1** Soft Actor-Critic for CartPole (code-equivalent)

---

**Initialize:** env, replay buffer  $\mathcal{D}$ , actor params  $\theta$ , critic params  $\phi_1, \phi_2$ , discount  $\gamma$ , temperature  $\alpha$ , batch size  $B$ , update interval  $K$

```

1 foreach environment step  $t$  do
2   observe state  $s$ ; sample  $a \sim \pi_\theta(\cdot | s)$ ; execute  $\rightarrow$ 
    $(s, a, r, s', d)$ ; store in  $\mathcal{D}$ 
3   if  $|\mathcal{D}| \geq B$  and  $t \bmod K = 0$  then
4     sample mini-batch  $(s, a, r, s', d)_B \subset \mathcal{D}$ 
5      $q^{\min}(s', a') \leftarrow \min(Q_{\phi_1}(s', a'), Q_{\phi_2}(s', a'))$ 
6      $V_{\text{soft}}(s') \leftarrow \sum_{a'} \pi_\theta(a' | s') [q^{\min}(s', a') -$ 
        $\alpha \log \pi_\theta(a' | s')]$ 
7      $y \leftarrow r + \gamma (1 - d) V_{\text{soft}}(s')$ 
8     for  $j \leftarrow 1$  to 2 do
9        $\phi_j \leftarrow \phi_j - \nabla_{\phi_j} (Q_{\phi_j}(s, a) - y)^2$ ;
10     $\theta \leftarrow \theta - \nabla_\theta \mathbb{E}_{a \sim \pi_\theta} [\alpha \log \pi_\theta(a|s) - q^{\min}(s, a)]$ ;
11  if  $d$  then
12    reset environment
    
```

---

### 2.4. Comparison - SAC & AC/A2C

As established, the SAC method is motivated over the actor-critic method. So we take a look at how exactly does SAC differ from methods like AC/A2C. The biggest addition is off-policy entropy regularization, AC/A2C both models maximize reward only. The entropy term is also implemented in calculating the soft state-value in SAC, compared to AC/A2C which directly computes value estimates directly. While, SAC is concerned with maximizing the reward plus entropy term together. Additionally, SAC being off-policy uses a replay buffer to sample transitions from for updates,

while AC/A2C is typically on-policy and updates after every sample. Lastly, for the critic network SAC utilizes two Q-functions to counter any overestimation in the prediction. Whereas AC/A2C typically use a single Q-function. The arguments are motivated by readings from both (Plaatt, 2022) & (Sutton & Barto, 2018)

### 3. Experiment

This section provides a detailed description of the implementation and subsequent experimentation of the SAC algorithm introduced in the theory section. A detailed breakdown of both the implementation and the experimental design is provided below, including a breakdown of performance metrics.

#### 3.1. implementation

For the implementation of the algorithm, there are 2 main sections. First, the algorithm itself, and second, the implementations of helper functions for estimating and storing performance metrics. As discussed in the theory section, there are two main implementations of the SAC algorithm: one is the standard SAC algorithm, and the other uses a double Q network. Both of these were implemented with mini-batching and variable update frequency. Mini-batch sampling from a larger relay buffer allows the samples to be correlated and smooths out the gradient estimates, which stabilizes the training process significantly. Equally important is the update frequency variable (here, every 10 environment steps), which determines how often forward/backward propagation is completed. This further stabilizes the models during training and also allows us to tune training time, striking a balance between training speed and training stability.

**SAC Standard** The core of the algorithm follows the standard actor-critic paradigm: you instantiate an Actor network to parameterize your policy and a Critic network to estimate state-action value functions. However, as discussed in the theory section, SAC introduces an entropy term, which serves as a form of regularization. When updating the critic, we compute the "soft"  $Q_{target}$  balanced by the hyperparameter  $\alpha$ . This hyperparameter determines the strength of the entropy term. The entropy term is calculated as simple Shannon's entropy:

$$\mathcal{H}(\pi(\cdot | s)) = - \sum_a \pi(a | s) \log \pi(a | s) \quad (6)$$

Higher  $\alpha$  favors a wider distribution of actions, thus encouraging exploration. The goal is to determine the value of alpha for the environment, which balances the action value against exploration.

**SAC double Q** To mitigate overestimation bias inherent in single-critic methods, we maintain two independent Q-networks (Critic 1 and Critic 2). During bootstrapping,

the target uses the minimum of the two soft-Q estimates. The "clipped" Target value serves as a safeguard to prevent positive overestimation of the values. This ensures more stability and decreases the effects of biased bootstrapping. The implementation simply duplicates the Q network and calculates both losses in parallel, after which it picks the smallest target of the two.

#### 3.2. Experiment Structure and Design

We evaluate the SAC algorithm on the cartpole environment. The goal of this environment is to balance a pole without letting it fall less than 30 degrees in each direction. Cartpole has a discrete action space consisting mainly of 2 actions  $a \in ["left", "right"]$ . The state space of the environment is a continuous 4-dimensional vector  $s \in [\text{Cart position, Cart velocity, Pole angle, Pole angular velocity}]$ . As for the reward, the environment rewards the agent 1 point for each step that the pole is within the 30-degree threshold. The environment has 2 main terminal states: one where the pole exceeds the angle threshold, and a second where 500 steps are reached.

We run each hyperparameter configuration 5 times to ensure the consistency of the results. Each run consists of a total of  $10^6$  steps, meaning each experiment completes  $5 \times 10^6$  steps. Two main metrics were used to measure performance. First, as a more substantial method, the trajectories of the average performance of each experiment were stored. This trajectory gives us an idea of how well this configuration performs over the 5 multiple runs. For a more empirical estimate of performance, the average reward per configuration was also calculated, serving as a measure of not only convergence but also stability and convergence speed.

Since the core paradigm of the algorithm is based on the Actor-critic method, the main hyperparameters were carried over from the best-performing AC method, which we evaluated in the last assignment. This includes but is not limited to specifically  $\gamma = 0.99$ , learning rate=0.001, batch size=64, and network architecture. The main hyperparameter of focus for this experiment was the  $\alpha$  hyperparameter, which determines the strength of entropy regularization. In total, performance was evaluated for four values of  $\alpha$  on the basic SAC implementation. The optimal value of  $\alpha$  was then carried over to the Double Q-learning implementation to evaluate whether it offered any performance improvement over standard SAC.

#### 3.3. Experimental results

After performing multiple runs on the chosen alpha values the average results are displayed in Table 1. Looking at the results, we notice that larger values for  $\alpha$  seem to yield

larger returns. With  $\alpha = [0.4, 0.8]$  yielding substantially larger average reward at 210 and 224, respectively. Whereas both smaller values for  $\alpha$  0.05 and 0.4 perform significantly worse with average rewards of 159 and 177, respectively. We do in fact see a substantial increase when implementing double q for the best  $\alpha$ . This configuration in particular reached an average reward of 284.

Table 1. Average training reward for different SAC configurations and  $\alpha$  values.

| Model        | $\alpha$ | Avg. Reward |
|--------------|----------|-------------|
| SAC Single Q | 0.4      | 210.28      |
| SAC Single Q | 0.8      | 224.38      |
| SAC Single Q | 0.2      | 177.67      |
| SAC Single Q | 0.05     | 159.58      |
| SAC Double Q | 0.8      | 284.10      |

There are two main contrasts at the core of this experiment. First, we compare each alpha value on the same model. To see the resulting training stability and development. By looking at the training reward trajectories over global training steps, we can determine how well each configuration adapts to the environment. Fig. 1 shows 4 different  $\alpha$  configurations on the standard SAC. These trajectories correspond to the first 4 average rewards in table 1. We notice now why the larger models have an overall larger return. When looking at the average trajectory, the 0.4 (blue) and 0.8 (orange) configurations manage to stabilize at a higher reward while the other alpha values, 0.05 and 0.2, struggle, especially 0.05, which later during the training settles at a much lower reward.

The second contrast focuses on the difference between the best standard SAC implementation versus the double Q implementation. As stated above, looking at Table 1, we see a substantial increase in average reward between SAC single Q 0.8 when compared to SAC Double Q 0.8, which reaches an average reward of 284. This performance increase is also visible when looking at individual trajectories as seen in Fig. 2. Here we see that the Double Q model (orange) has a slight advantage in convergence time, and paired with a more stable later stage than the standard SAC (blue) leads to a larger average return.

## 4. Discussion

Soft Actor Critic framework allows us to balance exploration and exploitation through the addition of an entropy term. This allows for more flexibility during training and gives us the ability to tune exploration through a more direct approach. However, a few extra additions are necessary for this algorithm to work appropriately. First, the addition of mini-batching allows for smoother gradients and more con-

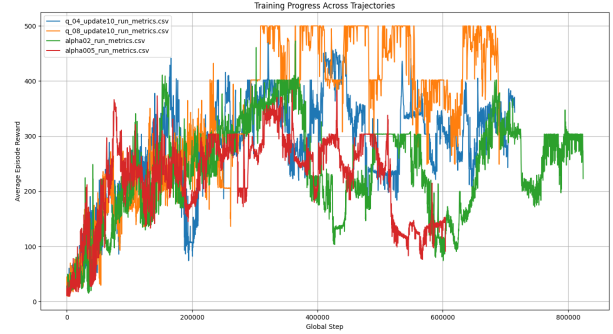


Figure 1. Trajectories of 4 different alpha values

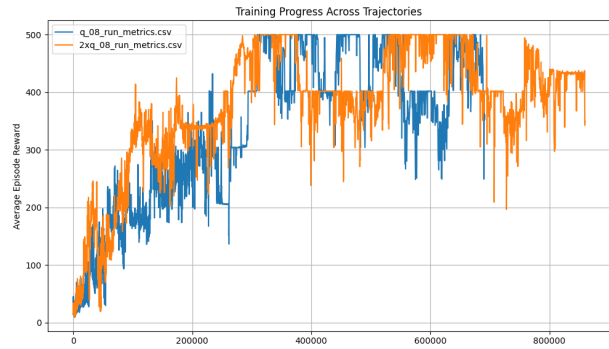


Figure 2. Trajectories of single and double Q SAC

sistent training trajectories. Second, this specific algorithm, when compared to Actor Critic, requires more computation. This need was mitigated somewhat by the addition of the update frequency, which allows us to tune the tradeoff between training stability and training speed.

Soft Actor-Critic introduces a new hyperparameter, the temperature  $\alpha$ , which governs the balance between exploration and exploitation. Selecting an appropriate  $\alpha$  is crucial yet non-trivial, as different environments may demand varying degrees of policy entropy. Therefore, with the inclusion of the entropy term, careful tuning of  $\alpha$  becomes a necessary—and potentially costly—step in algorithm design. Moreover, techniques such as entropy annealing or adaptive temperature adjustment can help automate this process and reduce the manual tuning burden.

### 4.1. Discussion of results

We see the importance of selecting an appropriate  $\alpha$  mirrored in the results. Larger entropy coefficients ( $\alpha = 0.4, 0.8$ ) consistently outperformed lower values, indicating that a softer policy (higher stochasticity) accelerates early exploration and leads to higher long-term returns. The best standard SAC was on the highest tested  $\alpha$  value at 0.8. When we compare this to the Double Q implementation as men-



tioned in the experiment section, we notice a substantial increase in performance. Trajectory plots show that Double Q converges slightly faster and exhibits lower variance in later stages, suggesting improved policy stability under the same hyperparameters.

There is one issue that we noticed with all the configurations. During the late stage, even the best-performing models are unstable and vary widely in performance. This could also be seen in what are visible flat sections in both Fig. 2 and 1. Since the trajectories are averaged over 5 runs, these sections imply that half of the runs were performing maximally and half of the runs were performing minimally. This consistent pattern would imply that exploration becomes detrimental in the latter stages, and an adaptive entropy approach could yield more stable results.

## 4.2. Limitations to our approach

It should be noted that this experimentation and the subsequent conclusions come with some severe limitations. The environment chosen to perform training for this algorithm is quite simplistic, thus undermining the potential improvements and advantages of the SAC. This is especially noticeable here due to the smaller and discrete action space taking the form of a binary action. Another limitation comes in the computational requirements. This algorithm is intrinsically more complex than the standard AC, thus requiring more computational resources, which limits our experimentation. As a consequence of this limitation, the resulting estimates suffer. Even though 5 runs do give us some idea of the performance, the lack of further replication limits the power we can leverage from our performance statistics. Furthermore, the heavy demand for computational resources limited the implementation of possible entropy annealing, which would require much more thorough testing of the  $\alpha$  decay functions.

## 5. Conclusion

Through the implementation and subsequent testing, the SAC algorithm allows us to leverage the entropy of the policy such that we can tune the balance between exploration and exploitation. The empirical results also show that an appropriate choice of  $\alpha$  is quite important and critical to the training process. We notice that larger values of  $\alpha$  seemingly accelerate the learning process and provide better results. Specifically,  $\alpha = 0.4$  and  $\alpha = 0.8$  achieved average episode rewards of 210.28 and 224.38, respectively, significantly outperforming the lower-alpha settings. We also see a subsequent increase in performance when the double Q network is added. As expected the clipping of the target value reduces overestimation and stabilizes training both increasing convergence speed and stability. This effect

is evident in the numerical results, where SAC Double Q with  $\alpha = 0.8$  achieved an average episode reward of 284.1, compared to 224.38 for SAC Single Q with the same alpha value.

Despite these gains, we observed an inherent instability in the second half of the training process. This stability is consistent with all tested configurations, suggesting that the introduction of entropy enforces exploration in later stages, increasing the number of non optimal. This introduces excessive stochasticity when the policy has converged. A remedy for this comes potentially in the form of a decaying  $\alpha$  which decreases entropy steadily throughout the training process.

Finally, while our findings on CartPole-v1 provide clear insights into SAC's mechanics, the simplicity of this environment and limited computational budget (five runs, fixed hyperparameters) constrain the generality of our conclusions. Future research should extend these experiments to higher-dimensional and stochastic domains, incorporate automated tuning of  $\alpha$  through decay functions, and ensure a set of more accurate estimates to appropriately fit models for general applications.

## References

- Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In Dy, J. and Krause, A. (eds.), *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pp. 1861–1870. PMLR, 10–15 Jul 2018. URL <https://proceedings.mlr.press/v80/haarnoja18b.html>.
- Haarnoja, T., Zhou, A., Hartikainen, K., Tucker, G., Ha, S., Tan, J., Kumar, V., Zhu, H., Gupta, A., Abbeel, P., and Levine, S. Soft actor-critic algorithms and applications, 2019. URL <https://arxiv.org/abs/1812.05905>.
- Plaat, A. *Deep Reinforcement Learning*. Springer Nature Singapore, 2022. ISBN 9789811906381. doi: 10.1007/978-981-19-0638-1. URL <http://dx.doi.org/10.1007/978-981-19-0638-1>.
- Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*. The MIT Press, 2018.